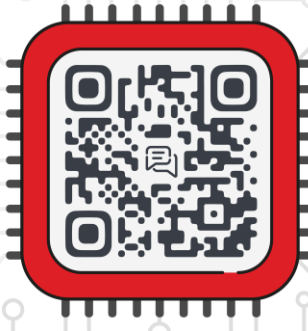




4



15



2250 sp

نظري

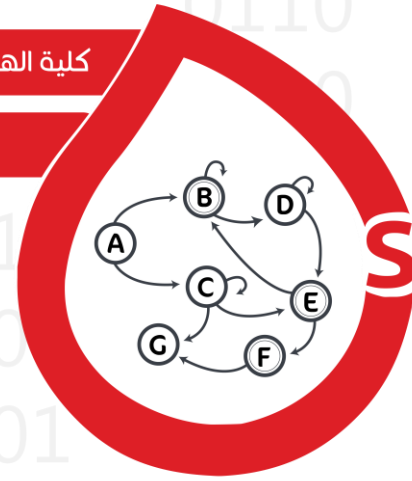
كلية الهندسة المعلوماتية

السنة الثالثة

التعابير المنتظمة

Regular expression

د. باسم قصيبة



اللغات الصورية

RB Informatics; 10/5/2026

محتوى مجاني غير مخصص للبيع التجاري

العمليات على الاوتومات :

في المحاضرة السابقة تعرفنا على القواعد (Grammar)  
وسنبداً محاضرتنا بتمرين عنها :

تمرين : هل كلا القواعد تشكل نفس الاوتومات؟

$G_1 :$

$x \rightarrow 0y \mid 0 \mid 1z$

$y \rightarrow 0x \mid 1y \mid 2$

$z \rightarrow 0z \mid 1x$

$G_2 :$

$A \rightarrow 0B \mid 1\epsilon$

$B \rightarrow 0A \mid 1F \mid \epsilon$

$C \rightarrow 0C \mid 1A$

$D \rightarrow 0A \mid 10 \mid \epsilon$

$E \rightarrow 0C \mid 1A$

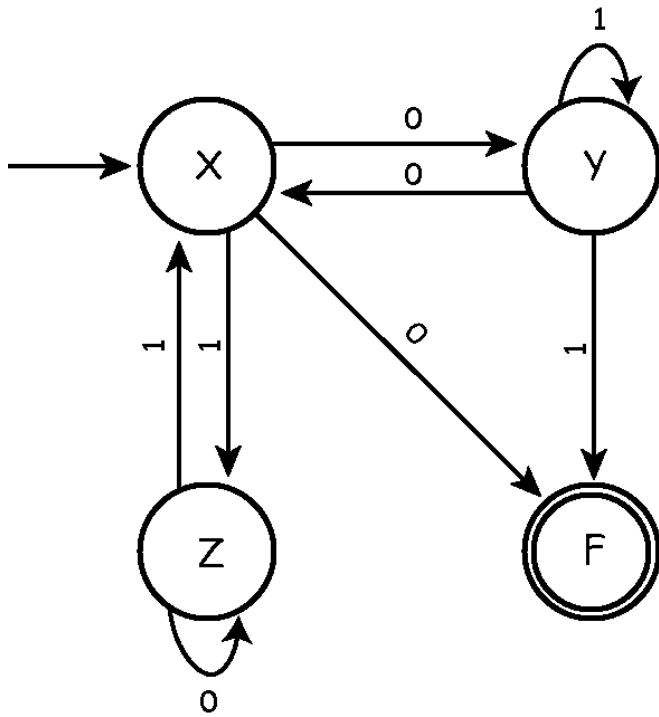
$F \rightarrow 0A \mid 1B \mid \epsilon$

فكرة الحل:

نرسم الأوتومات لمجموعة القواعد الأولى ( $G_1$ ) ومجموعة القواعد الثانية ( $G_2$ ) ثم نقارنهم .

➡ الحل:

نبدأ بمجموعة القواعد الأولى ( $G_1$ ) و نرسم الاوتومات المقابل لها.



تذكرة:

\* اعتبرنا الحالة الأولى هي الحالة الابتدائية لأنه لم يحدد لنا الحالة الابتدائية.

\* أضفنا حالة نهائية جديدة لأنه لا يوجد حالة نهائية محددة من الأوتومات.

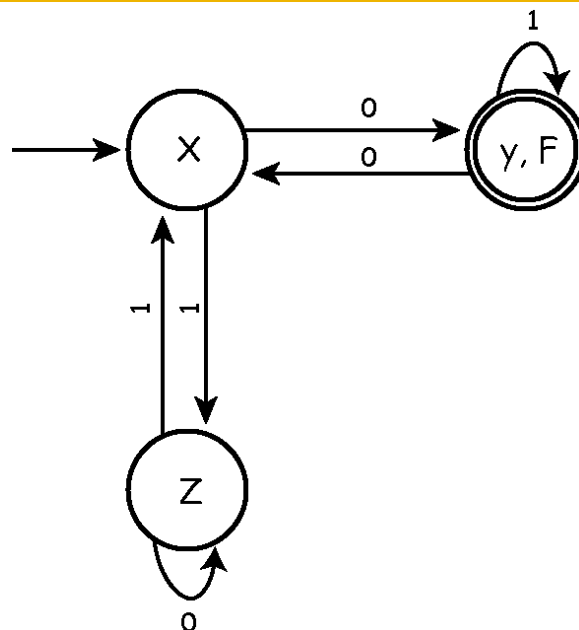
نحول الاوتومات السابق إلى DFA :

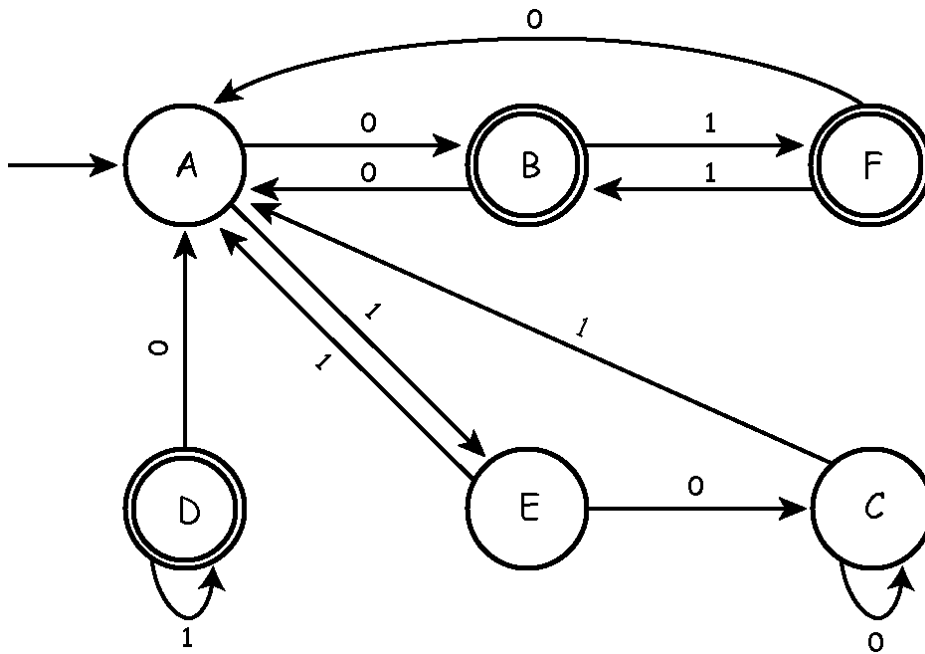
|          | 0      | 1      |
|----------|--------|--------|
| {x}      | {y, F} | {z}    |
| * {y, F} | {x}    | {y, F} |
| {z}      | {z}    | {x}    |

حالات جديدة

يوجد حالة نهائية

نرسم الاوتومات :





1. اخترنا الحالة الأولى هي الحالة الابتدائية لأنه لم يحدد لنا الحالة الابتدائية.
2. لم نضف حالات جديدة لوجود  $\epsilon$  ضمن القواعد, و اعتبرنا الحالات الموجودة فيها  $\epsilon$  حالات نهائية.

الآوتومات السابق عبارة عن DFA ولا داعي لتحويله, ولكن يبقى لنا أن نختصر الآوتومات:

## خطوات اختصار الآوتومات:

1. اختصار الحالات التي لا يمكن الوصول لها

نلاحظ وجود الحالة D (لا يوجد أي سهم يدخل للحالة)

2. اختصار الحالات الميتة Dead State

نلاحظ عدم وجود حالات ميتة في الآوتومات السابق

3. اختصار الحالات المتكافئة:

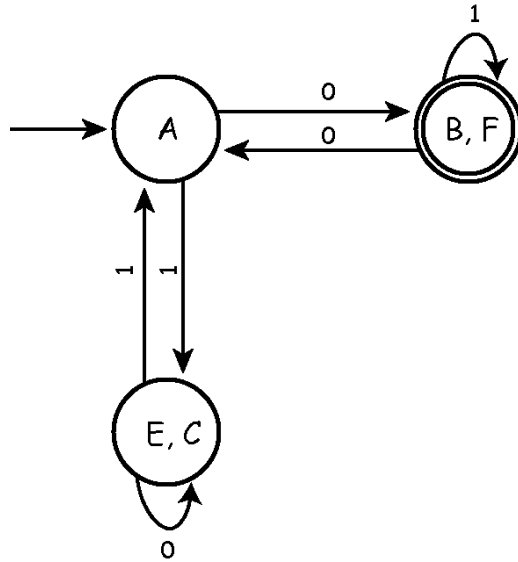
| A                  | E                  | C |
|--------------------|--------------------|---|
| $\delta(A, 0) = C$ | $\delta(A, 1) = E$ |   |
| $\delta(E, 0) = B$ | $\delta(E, 1) = A$ |   |
| $\delta(C, 0) = C$ | $\delta(C, 1) = A$ |   |

الحالتان A, C تصبان في نفس المجموعة عند ال 0 و ال 1

| B                  | F                  |
|--------------------|--------------------|
| $\delta(B, 0) = A$ | $\delta(B, 1) = B$ |
| $\delta(F, 0) = A$ | $\delta(F, 1) = F$ |

عند ال 0 تصبان في نفس المجموعة

الحالات عند ال 1 تصبان في نفس المجموعة



بالتالي يصبح الاتومات :

بالمقارنة بين الاتوماتين نلاحظ أن A يكافئ x ، وأن B, F يكافئ y, F ، وأن E, C يكافئ z .  
بالتالي: نعم , القواعد تولد نفس الاتومات.

## Regular expressions التعبيرات المنتظمة

التعبير المنتظم : هو طريق صحيح من الحالة الابتدائية إلى الحالة النهائية.

وكتابة التعبيرات المنتظمة تحتاج لقواعد من الشكل:  $A \rightarrow aA \mid b$

- القاعدة السابقة تقبل صفر من a , و b واحد
- أو عدد a لا نهائي من المرات و ثم b
- وهي مكافئة للقاعدة  $A \rightarrow a^* b$  (ال \* تدل على إمكانية وجود a عدد لانهائي من المرات أو عدم وجوده)

بمعنى آخر التعبير المنتظم هو شكل آخر لكتابة القواعد.

مثال:

تحويل القواعد S إلى تعبير منتظم، حيث لدينا القواعد التالية:

$$S \rightarrow 0A \mid 0 \mid 1B$$

$$A \rightarrow 1A \mid 1$$

$$B \rightarrow 0B \mid 1S$$

الحل:

نكتب A بشكل تعبير منتظم :  $A = 1^* 1$  (دائما عند وجود الحالة نفسها ضمن القاعدة نستطيع وضع \* عوضها )

ونكتب B بشكل تعبير منتظم :  $B = 0^* 1S$

و نكتب S بشكل تعبير منتظم بتعويض ال A و ال B :

$$S = 0(1^* 1) + 0 + 1(0^* 1S)$$

نعيد ترتيب كتابة التعبير :

$$S = 1(0^* 1S) + 0 + 0(1^* 1)$$

نخرج ال S خارج الأقواس

ندخل الصفر لداخل القوس

$$S = (1 \ 0^* \ 1) S + (0 \ 1^* \ 1 + 0)$$

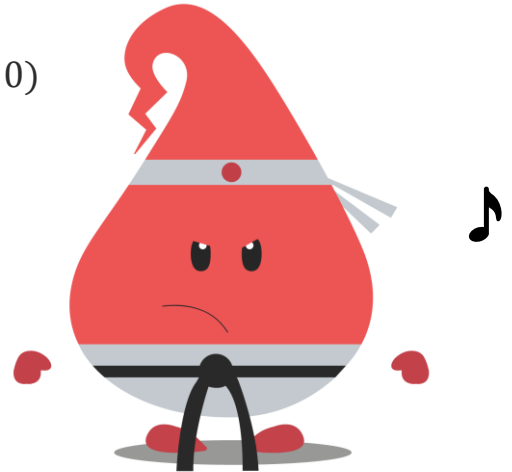
نطبق القاعدة: عند تكرار الحالة نضع \*

$$S = (1 \ 0^* \ 1)^* + (0 \ 1^* \ 1 + 0)$$



Just beat it, beat it.....

No one wants to be defeated ^^



تذكرة:

ما معنى الإغلاق على مجموعة؟

- مجموعة الأعداد الطبيعية مغلقة (Closed) على عملية الجمع، لأن عملية الجمع بين أي عددين من مجموعة الأعداد الطبيعية يعطي عدداً من ذات المجموعة.
- مجموعة الأعداد الطبيعية غير مغلقة على عملية الطرح، لأن عملية الطرح بين عددين من مجموعة الأعداد الطبيعية قد يعطي عدداً سالباً (من خارج المجموعة).
- مجموعة الأعداد الصحيحة مغلقة على عمليات الجمع والطرح والضرب، وغير مغلقة على عملية القسمة.

اللغات المنتظمة مغلقة على العمليات التالية:

1. الاجتماع (Union)
2. التقاطع (Intersection)
3. الوصل (Concatenation)
4. الفرق (Difference)
5. المتمم (Complement)
6. النجمة (Kleene Star)
7. المعكوس (Reverse)

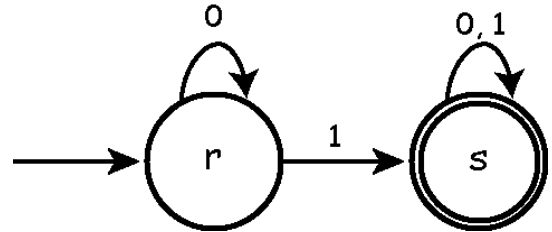
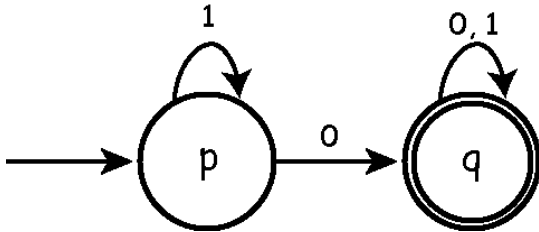
### التقاطع:

ينتج لاتومات يقبل الكلمات التي يقبلها اللاتومات الأول والثاني معاً.

مثال:

اللاتومات الثاني :

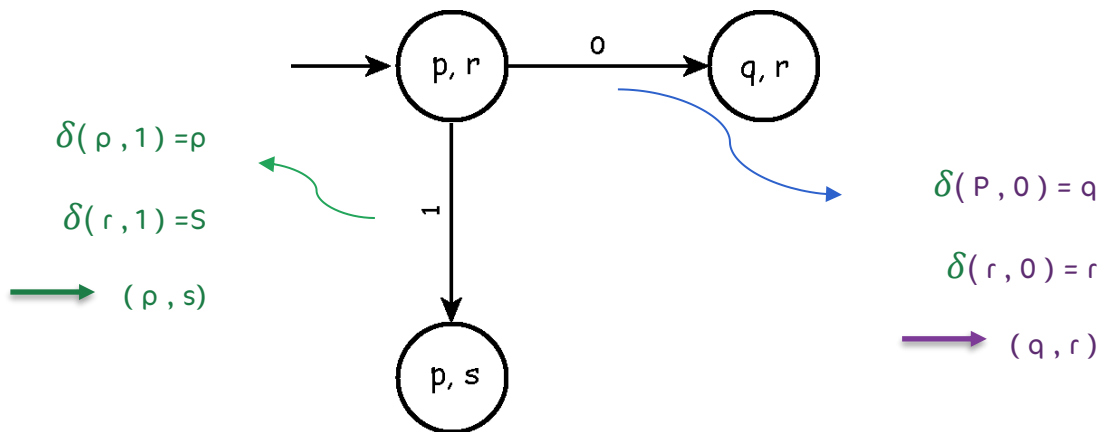
اللاتومات الأول :



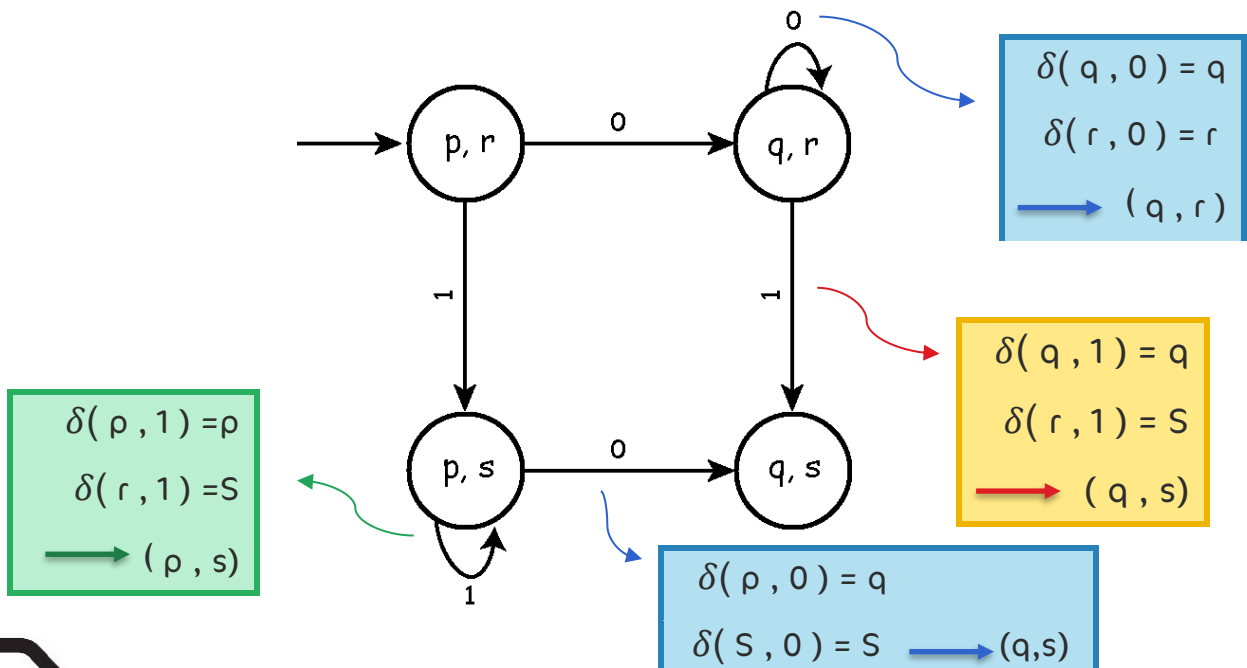
يقبل أي كلمة تحتوي على الرقم 0 على الأقل

يقبل أي كلمة تحتوي على الرقم 1 على الأقل

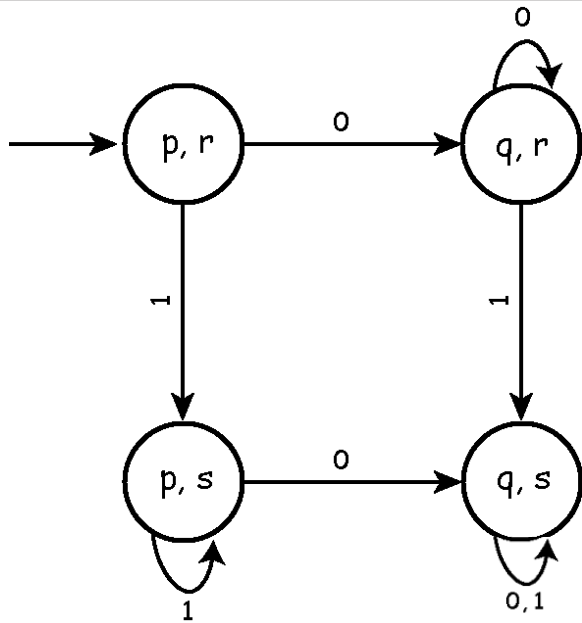
■ ندمج الحالات الابتدائية ونضع انتقالاتها عند الدخل 0 والدخل 1 :



نكمل انتقالات الحالات عند الدخل 0 والدخل 1:



نكمل الانتقالات عند الحالة  $q, s$  :



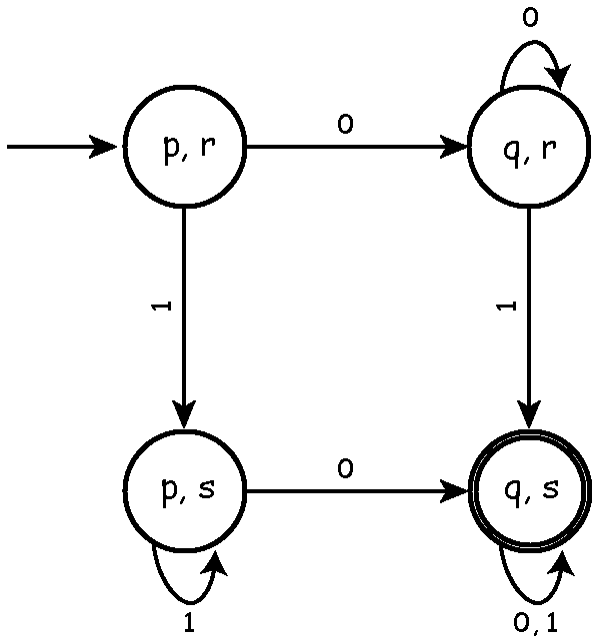
$$\delta(q, 0/1) = q$$

$$\delta(r, 0/1) = s$$

→  $(q, s)$

الحالات النهائية في الاوتومات الجديد هي الحالات التي تحوي على حالة نهائية في الاوتومات الأول و الاوتومات الثاني معاً (شرط التقاطع)، وهي الحالة  $(q, s)$  فقط.

بالتالي يصبح الشكل البنائي للاوتومات :



الاوتومات الناتج بصير اوتومات يقبل الكلمات التي تحوي على الرقم 1 والرقم 0 مرة على الأقل

**للتحقق** نجرّب كلمة مثل 0000:

$$(p, r) \xrightarrow{0} (q, r) \xrightarrow{0} (q, r) \xrightarrow{0} (q, r) \xrightarrow{0} (q, r)$$

لم نصل إلى حالة نهائية، فالأوتومات الناتج عن التقاطع لم يقبل الكلمة 0000، وبالتالي أحد الاوتوماتين على الأقل لن يقبل هذه الكلمة.

(لأنه كما شرحنا الأوتومات الأول يطلب وجود الرقم 1 على الأقل لكي يقبل الكلمة)

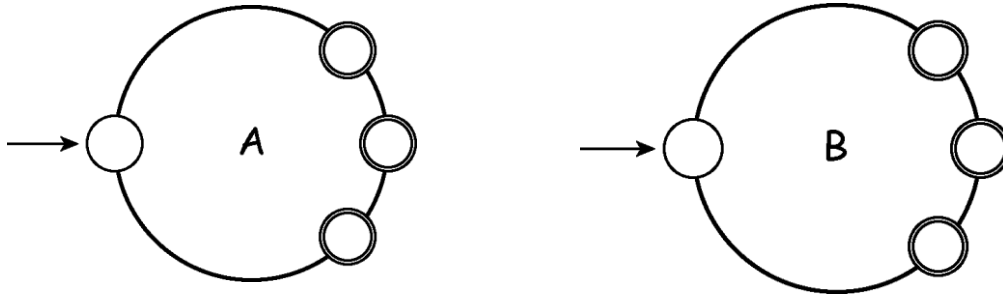
وأيضاً العكس صحيح، الكلمات التي لم يقبلها أحد الاوتوماتين لن يقبلها الاوتومات الناتج عن التقاطع.

## الاجتماع:

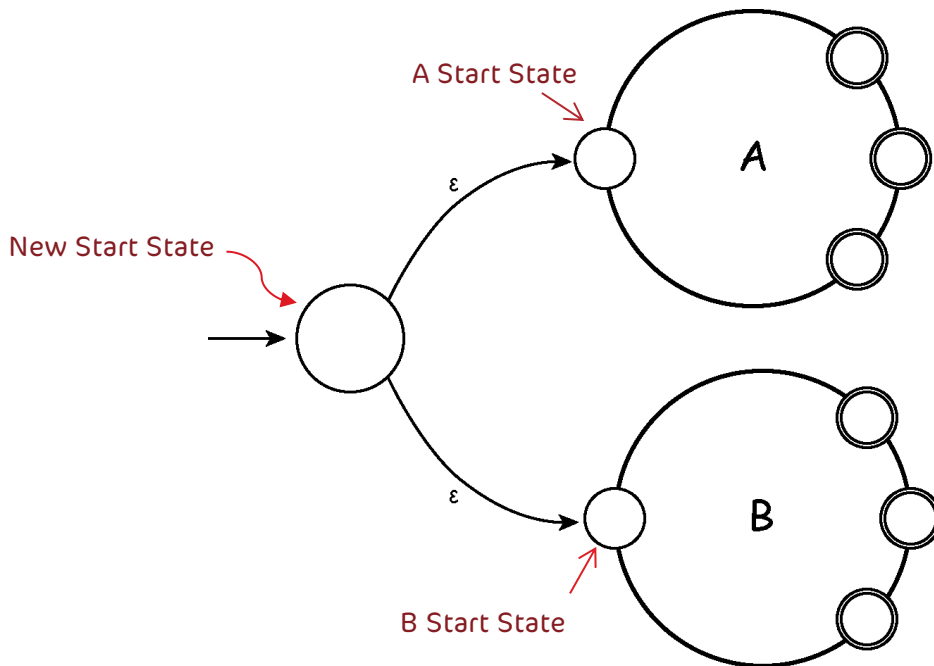
ينتج اوتومات يقبل الكلمات التي يقبلها أحد الاوتوماتين الأول أو الثاني.

مثال:

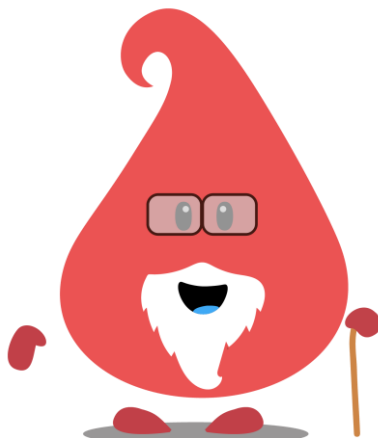
ليكن لدينا الاوتوماتين A و B :



يكون اجتماع الاوتوماتين بوضع حالة ابتدائية جديدة ( New Start State ) ، لها انتقال بقيمة  $\epsilon$  (epsilon) إلى كل من الحالتين الابتدائيتين لل A و B .



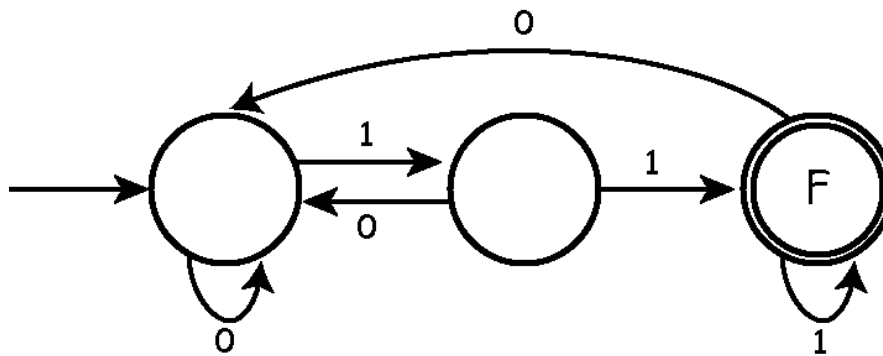
You don't have to be perfect  
to be amazing



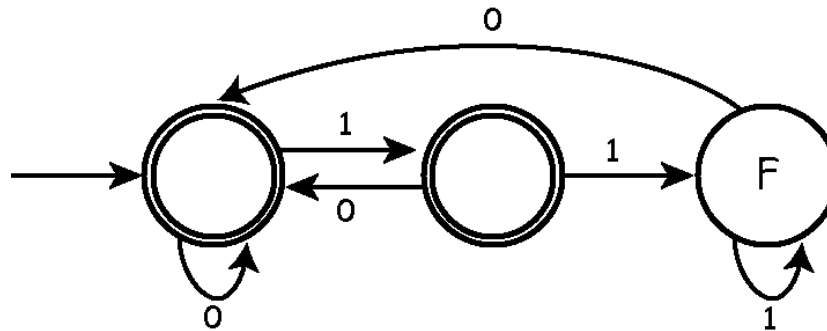
## المتمم:

نحصل عليه بجعل الحالات العادية نهائية والحالات النهائية تجعلها عادية.  
يجب أن يكون الاوتومات أولاً محولاً إلى Complete DFA أي بدون حذف حالات ميتة (Dead States) ، إن وجدت المتمم يعني "العكس تماماً".  
إذا كان لدينا أوتومات يقبل لغة معينة (مثلاً يقبل الكلمات التي تنتهي بـ 1)، فإن الأوتومات "المتمم" له سيرفض هذه الكلمات، ويقبل كل شيء آخر (أي يقبل الكلمات التي لا تنتهي بـ 1)

**مثال :**



المتمم هو:



## الفرق :

ينتج اوتومات يقبل الكلمات التي يقبلها الاوتومات الأول ولا يقبلها الاوتومات الثاني، وتعطى بالعلاقة:

$$L - M = L \cap \overline{M}$$

المتمم

التقاطع

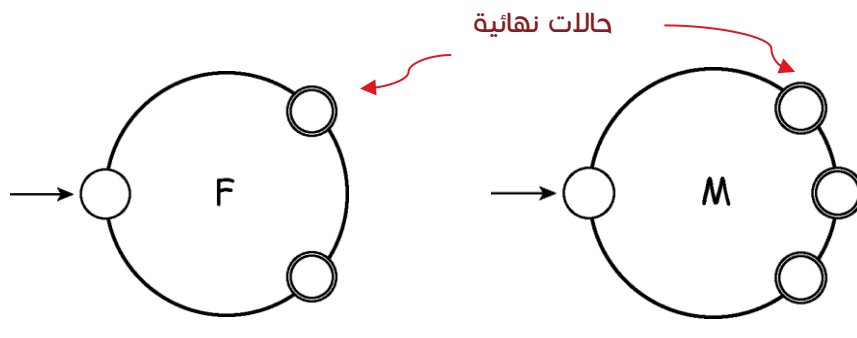
وبذلك نحسب المتمم للاوتومات الثاني ثم نحسب التقاطع ← فينتج الاوتومات المطلوب.

## الوصل:

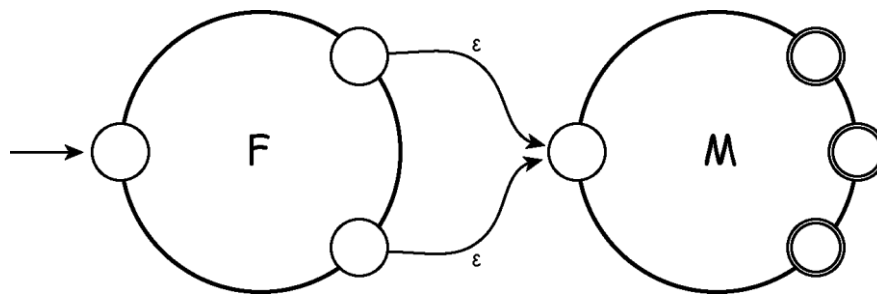
ينتج اوتومات يقبل الكلمات المقبولة في الاوتومات الأول متبوعة بالكلمات المقبولة في الاوتومات الثاني على الترتيب (لأن  $F$  موصول بـ  $M$  يختلف عن  $M$  موصول بـ  $F$ )  
بفرض لدينا اوتومات  $L$  واوتومات  $M$  ، فيرمز للوصل بينهما بـ  $L.M$

تكون من خلال وصل الحالات النهائية للاوتومات الأول مع الحالات الابتدائية للاوتومات الثاني بانتقال  $\epsilon$  وتحويل الحالات النهائية السابقة للاوتومات الأول لتصبح حالات عادية ، وتبقى حالات الاوتومات الثاني النهائية هي النهائية للأوتومات المشترك.

### مثال :



ف تصبح :



لدينا الأوتومات  $F$  (حالتان نهائيتان يمينًا)، والأوتومات  $M$  (حالة بداية يسارًا، حالات نهائية يمينًا)  
لاحظ خروج سهمين من نهايات  $F$  ودخولهما إلى بداية  $M$

ملاحظة دقيقة: حسب القاعدة، تفقد حالات  $F$  المزدوجة في الرسم السفلية حالاتها النهائية نظرياً (الدخول الكلي للأوتومات أصبح من  $F$  ، والخروج الكلي (القبول) أصبح من  $M$  فقط.

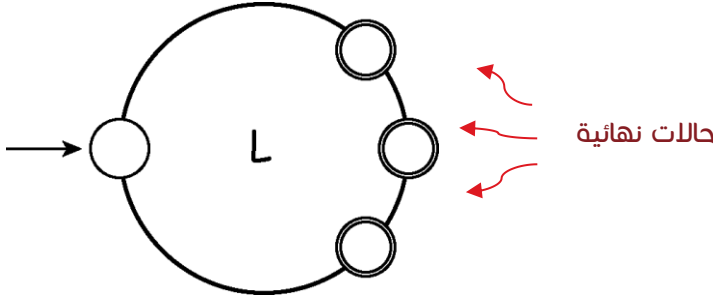
## (Kleene Star) K-star

عملية K-star هي عملية وصل متكرر لنفس الاوتومات.  
 بفرض لدينا الاوتومات  $L$  ونريد وصله مرتين يرمز لها بـ  $L^2 = L . L$   
 وفي حال نريد وصله 3 مرات يرمز لها بـ  $L^3 = L . L . L$  وهكذا...  
 ونلاحظ أن :

$$L^0 = \{\epsilon\} , \quad L^1 = L , \quad L^2 = L . L , \quad L^3 = L . L . L$$

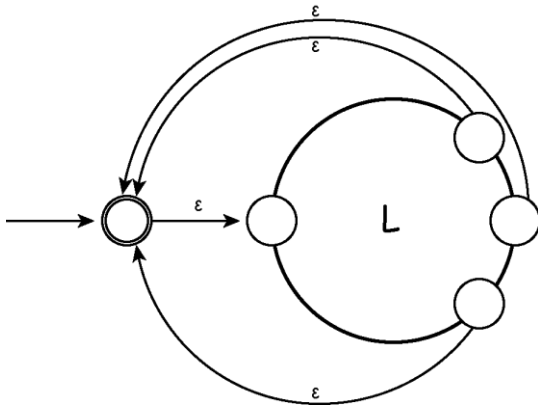
**مثال:**

نريد تطبيق عملية K-star على الاوتومات  $L$  :



$L^0 = \{\epsilon\}$  : ماذا لو كررنا الأوتومات صفر مرة؟

النتيجة هي مجموعة تحتوي على الكلمة الفارغة فقط (إبسيلون  $\epsilon$ ) هذا يعني استنتاجاً أي أوتومات يُطبق عليه عملية نجمة كلين ( $L^*$ ) يجب، إجبارياً، أن يقبل الكلمة الفارغة.



1. نضيف حالة ابتدائية جديدة ونجعلها نهائيةً .
2. نصل هذه الحالة بالحالة الابتدائية السابقة بانتقال  $\epsilon$
3. نجعل الحالات النهائية السابقة حالات عادية.
4. نصل جميع الحالات النهائية السابقة بالحالة الابتدائية الجديدة بانتقال  $\epsilon$

😊 نلاحظ:

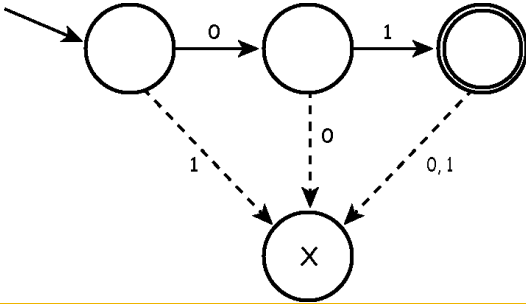
- الحالة  $L^0$  محققة لأنه بدون أي دخل، فالكلمة مقبولة لوجود الحالة النهائية في البداية.
- الحالة  $L^1$  محققة لأنه عند الوصول للحالات النهائية السابقة نكون وصلنا إلى حالة نهائية بفضل وجود انتقال  $\epsilon$ .
- الحالات  $L^{2,3,\dots}$  محققة لأنه عند نهاية كلمة مقبولة من  $L$  نستطيع وصلها بكلمة مقبولة أيضاً من  $L$  لعدد لا نهائي من المرات.

## المعكوس:

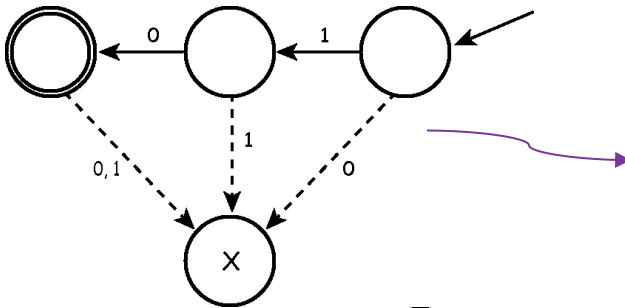
ينتج اوتومات يكون فيه جميع الحالات النهائية ابتدائية وجميع الحالات الابتدائية نهائية بالإضافة إلى عكس اتجاه جميع الأسهم (الانتقالات).

عند وجود أكثر من حالة نهائية في الاوتومات الأصلي، نضيف حالة جديدة واحدة ونجعلها هي الابتدائية ونصلها مع جميع الحالات النهائية السابقة بانتقال  $\epsilon$  ثم نقوم بعكس الأسهم .

مثال :

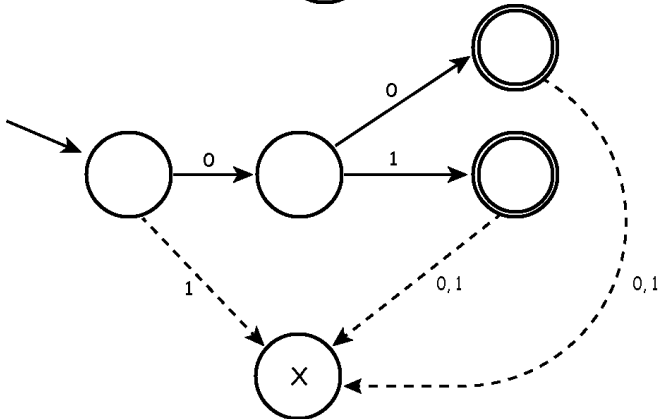


تصبح :

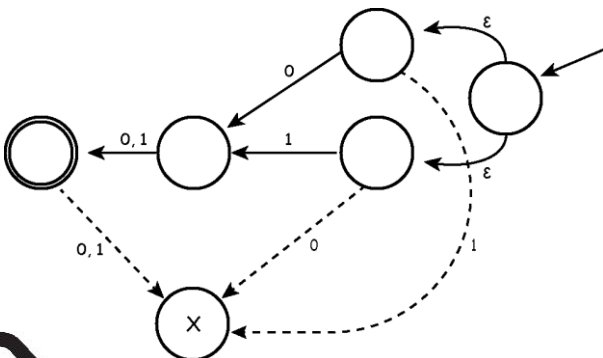


ملاحظة: لا داعي لقلب اتجاه الأسهم عند الحالات الميتة فقط نبذل أحرف الانتقالات

مثال 2 :



فتصبح



## تحويل التعبير المنتظم Regular Expression إلى اوتومات

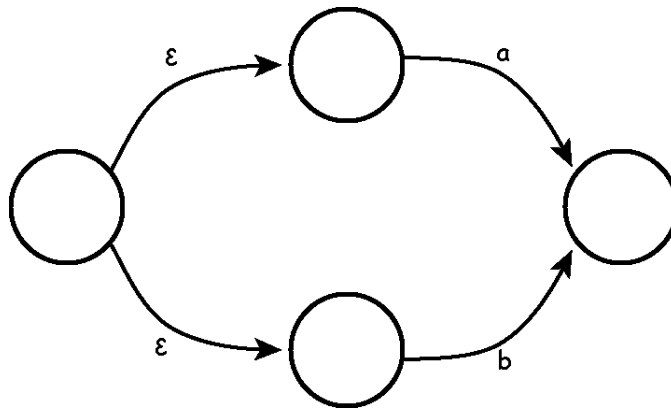
**تمرين:**

ليكن لدينا التعبير المنتظم التالي:

$$a \cdot (a + b)^* \cdot b \cdot b$$

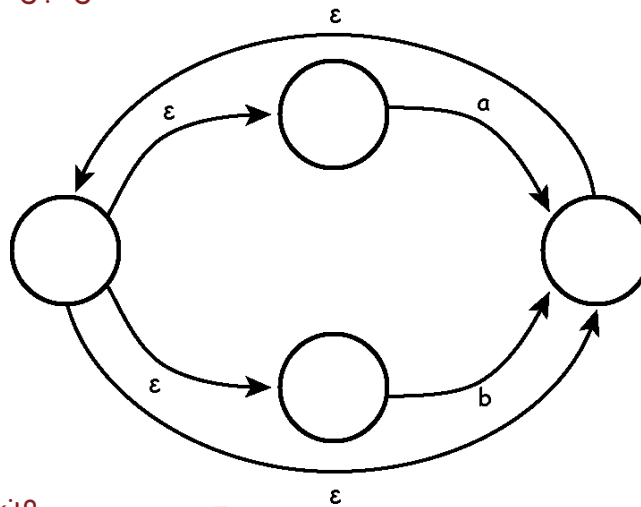
لحل هذا التمرين نبدأ برسم الاوتومات قطعة قطعة ثم نركبها بالترتيب:

1. نرسم أولاً الجزء الخاص بـ  $(a + b)$  :



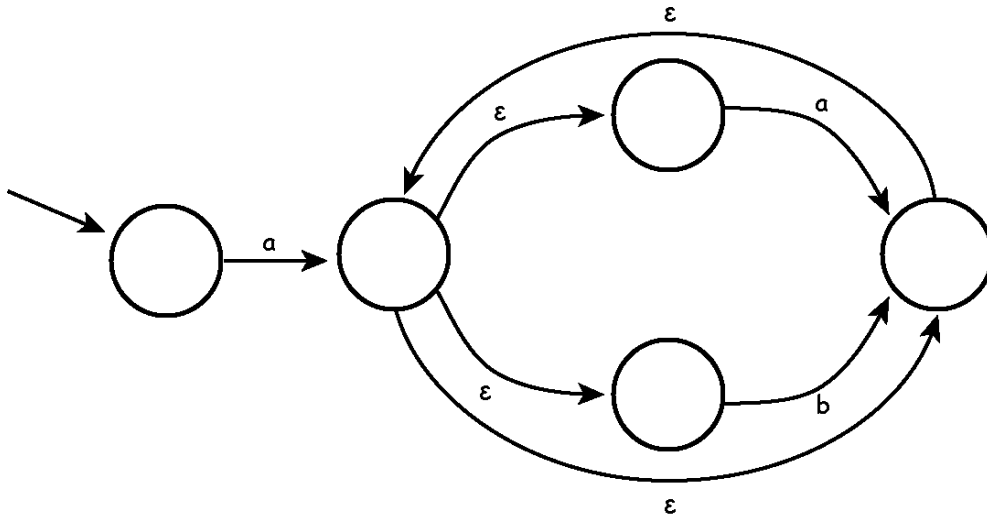
2. نضيف عليها الـ  $*$  لتصبح  $(a + b)^*$

من أجل أن تقبل عدد لا نهائي

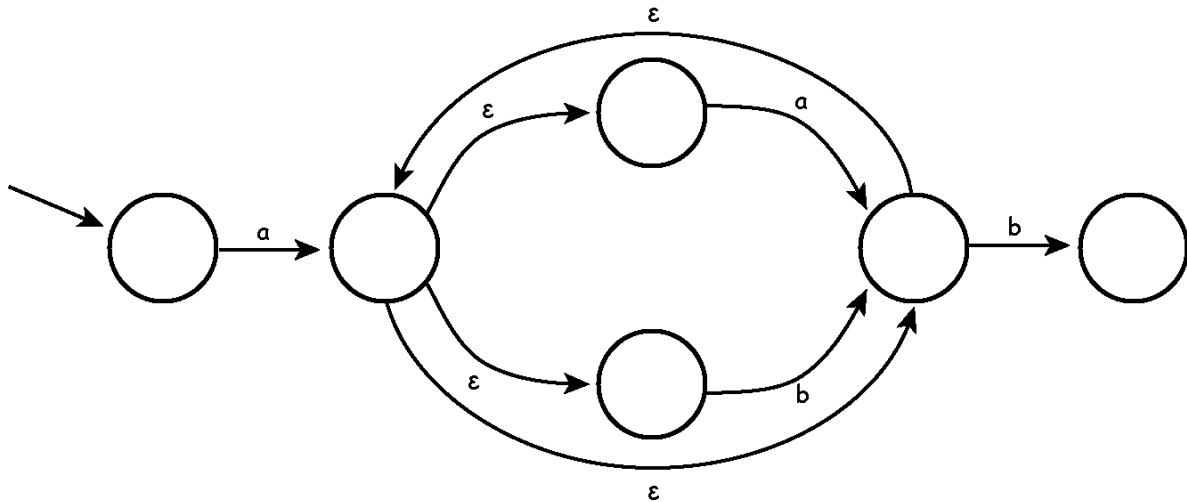


من أجل أن تقبل عدد 0

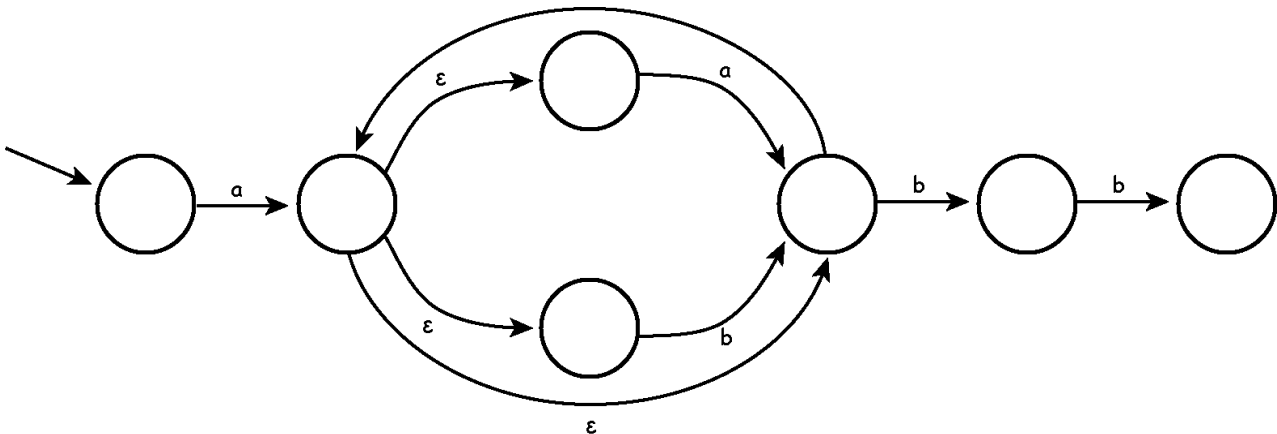
3. نصل a بها مع مراعاة الترتيب لأنه مهم  $a.(a + b)^*$



4. نصل b :  $a.(a + b)^*.b$



5. ثم نصل بها b الأخيرة فيصبح الناتج النهائي :  $a.(a + b)^*.b.b$



اللغات المنتظمة ( Regular Languages )

لدينا بعض القواعد في اللغات المنظمة :

1.  $L + M = M + L$  الجمع تبديلي ( Commutative )
  2.  $(M + N) + L = M + (N + L)$  الجمع تجميعي ( Associative )
  3.  $(LM) N = L(MN)$  الضرب تجميعي
  4.  $L(M + N) = LM + LN$  الخاصية التوزيعية (Distributive)
  5.  $(L^*)^* = L^*$
  6.  $\phi^* = \epsilon$
  7.  $\epsilon^* = \epsilon$
  8.  $L^+ = L \cdot L^*$  حاليا ما رح نستخدمهم , منشوفهم بالترجمات
  9.  $L^? = L \cdot \epsilon$
  10.  $(L + M)^* = (L^* \cdot M^*)^*$
  11.  $\phi + L = \phi + L = L$
  12.  $\epsilon \cdot L = L \cdot \epsilon = L$
  13.  $\phi \cdot L = L \cdot \phi = \phi$
  14.  $(\epsilon + R)^* = R^*$
  15.  $(\epsilon + R)R^* = R^*$
- قوانين مهمة جدا!!!



*The end*